

Retrieval-Augmented Generation (RAG) Flow in Aagosh AI

This document details how the RAG (Retrieval-Augmented Generation) pipeline is implemented in the Aagosh AI backend. The primary goal of this system is to ground the AI's parenting advice in authoritative, age-appropriate knowledge sources while keeping the application lightweight and free from external vector database dependencies.

1. Triggering the RAG Pipeline

The RAG process begins when a user sends a message in a coaching conversation via the `ParentingCoachService` (`app/coach/service.py`).

- The service intercepts the user's message and initiates the context assembly phase before hitting the LLM.

2. Context Assembly (`ContextAssemblyService`)

Located in `app/services/context_assembly_service.py`, the `build_context` method is responsible for gathering all deterministic data and retrieved knowledge:

- **Child Profile:** Calculates the exact age of the child and retrieves their strengths, challenges, and active parenting goals.
- **Recent Activity:** Pulls recent check-ins and behavior events from the database.
- **Knowledge Retrieval:** Triggers the

`KnowledgeRetrievalService` with the user's query and a strict age filter based on the child's calculated age. If no results are found, it performs a fallback search without the age filter.

3. Embedding and Retrieval (`KnowledgeRetrievalService`)

Located in `app/knowledge/retrieval.py`, this service acts as the bridge to the vector store:

- It uses an embedding provider to convert the user's text query into a numerical vector representation.
- It queries the `FileVectorStore` using the generated embedding, the original text query, and the age filters.

4. The Vector Store (`FileVectorStore`)

Located in `app/knowledge/vector_store.py`, Aagosh uses a local JSON-based vector store (`backend/data/knowledge_store.json`) to store and search through knowledge chunks.

- **Filtering:** It pre-filters chunks based on metadata constraints like `age_min`, `age_max`, `category`, and `tags`.
- **Hybrid Search:** It calculates the Cosine Similarity between the query vector and the stored document vectors. It also applies a keyword-overlap boost (checking for common words between the query and the chunk's content/tags) to improve precision.
- **Scoring:** Results are scored, sorted, and the top-K results are returned to the context assembler.

5. Prompt Construction and LLM Execution

Back in the `ParentingCoachService`:

- The retrieved knowledge and deterministic data are sanitized by the `LLMContextSanitizer` to remove sensitive PII.
- The `ParentingPromptBuilder` injects this sanitized context, the retrieved knowledge, and the conversation history into the system prompt.
- The LLM provider processes this rich, grounded prompt to generate a response.

6. Safety Validation and Citations

(`ParentingSafetyValidator`)

Before the response is returned to the user:

- The `ParentingSafetyValidator` checks the LLM's output against the retrieved knowledge to ensure it didn't hallucinate or violate safety constraints.
- It programmatically generates citations (Source References) linking the LLM's advice back to the specific chunks of knowledge that were retrieved from the vector store.